ie: "to zero a register"

eax := general purpose register

xor := bitwise exclusive or

true only when inputs differ

xor <reg>, <reg>

∴ xor eax, eax → sets eax to false ie 0000

zeros out eax ⇒ zero overhead

eax → false!

obfuscation OR preset value ??

ex

lea := load effective address

lea <reg32>, <mem>

∴ lea rbx, [0]

(local var, caller-owned)

gen-purpose regs

x86-64 regs

rcx = 0 → 0--=-1 (no flags affected)

N ↓ [rcx]--

0xFF....F ↳ ie loop loops ie crash...

address of 0 placed in ~~rbx~~ RBX

∴ zeros out rbx...

loop $

① decrements rcx (loop register)

② jumps to provided label if register not zero

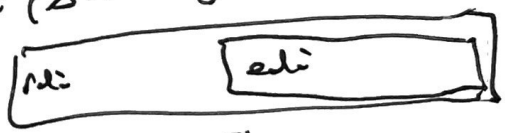
mov rdx, 0 → move the value of 0 into rdx

ie manually set the 64-bit rdx reg. to 0s

doesn't touch the status flags

and esi, 0 → zeros out esi (& ∴ rsi) + sets ZF = 1 (zero flag)

sub edi, edi → zeroes out ~~edi~~ edi (& ∴ rdi)



push 0 → puts 0 to stack

① decrement stack ptr by 8 bytes (rsp) + ② writes 0 to memory address stored in rsp

pop rbp → reads 0 just pushed & loads it into rbp + adds 8 to rsp to clean stack